

RAG-Projekt — Betrieb

Installation, Kommandozeilen und typische Abläufe

Voraussetzungen

Das Projekt verlangt Python 3.12 oder neuer. Verwalter wird das Projekt mit uv; die Abhängigkeiten stehen in pyproject.toml und werden über uv.lock festgeschrieben. Für die RAG-Anwendung braucht es einen Zugangsschlüssel bei OpenRouter, für die drei Vorverarbeitungsstufen dagegen weder Netz noch Schlüssel.

Einrichtung

Nach dem Klonen des Repositoriums werden die Abhängigkeiten installiert. Der Entwicklungsmodus zieht zusätzlich die Testwerkzeuge.

Schritt	Befehl
Abhängigkeiten installieren	uv sync
Mit Testwerkzeugen	uv sync --group dev
Umgebung aktivieren (Linux/macOS)	source .venv/bin/activate
Umgebung aktivieren (PowerShell)	.\.venv\Scripts\Activate.ps1

Einstiegspunkte

pyproject.toml definiert sechs Skripte. Jedes zeigt auf eine eigene CLI im Paket.

Befehl	Ziel	Zweck
uv run rag-project	app/main.py	Konfiguration prüfen, Modellkatalog zeigen
uv run chatbot	app/main_chatbot.py	Terminal-Chatbot mit Verlauf
uv run loading	app/main_loading.py	Dokumente laden und melden
uv run cleaning	app/main_cleaning.py	Text bereinigen
uv run chunking	app/main_chunking.py	Dokumente chunken

Befehl	Ziel	Zweck
uv run rag	app/rag_app.py	vollständige RAG-Anwendung

Projekt-CLI

Der Einstieg rag-project dient der Prüfung von Setup und Modellwahl.

Aufruf	Wirkung
uv run rag-project	kurzer Statusbericht
uv run rag-project --check	Keys vorhanden, Modell auflösbar, Base-URL
uv run rag-project --models	gesamten Modellkatalog zeigen
uv run rag-project --models openai	Katalog auf einen Anbieter gefiltert
uv run rag-project --show-config	aktive Einstellungen, Keys nur als Fingerabdruck
uv run rag-project --model openai:gpt-4o-mini	Modell für diesen Lauf erzwingen

Der Statusbericht zeigt Provider, Modell und Base-URL. Der Schalter --check löst zusätzlich das Ziel auf und gibt Provider, Modellkennung, Modelltyp, Base-URL und Parameter aus. Der Schalter --show-config gibt describe_settings() aus; Schlüssel erscheinen dort nie im Klartext, sondern nur als „set (n chars)“.

Chatbot

Der Chatbot hält einen Nachrichtenverlauf als LangChain-Liste: System-Nachricht an Position 0, danach abwechselnd Nutzer und Antwort. Der Verlauf wird erst nach dem letzten Streaming-Stück ergänzt, damit ein abgebrochener Stream keine halbe Antwort hinterlässt.

Befehl	Wirkung
/help	Übersicht der Befehle
/system <Text>	System-Prompt ersetzen, Verlauf leeren
/show	aktuellen System-Prompt zeigen
/reset	Verlauf löschen, System-Prompt behalten

Befehl	Wirkung
/model	aktives Modell und Provider zeigen
/mode stream invoke	Ausgabemodus wechseln
/exit oder /quit	beenden

Der Ausgabemodus startet als stream und lässt sich mit --no-stream auf invoke stellen. Im Stream werden Textstücke ausgegeben, sobald sie eintreffen; invoke liefert die vollständige Antwort auf einmal.

Die drei Vorverarbeitungs-CLIs

loading, cleaning und chunking haben dieselbe Bedienung: Positionsargumente sind Quellpfade, Optionen steuern das Verhalten, und ohne Argument greifen die Pfade aus der .env. Jede der drei Anwendungen nimmt zusätzlich netzbasierte Quellen an, erkennbar am Präfix src:. Fortschritt geht nach stderr, Ergebnisse nach stdout.

Aufruf	Wirkung
uv run loading --preview 200	laden, Vorschau der ersten Zeichen
uv run loading --show-paths	aufgelöste Pfade vor dem Laden zeigen
uv run cleaning --steps	zeigen, welcher Schritt wie viel bewirkt hat
uv run chunking --compare	alle Strategien bei gleichem Zielwert vergleichen
uv run chunking --handbook	Trennzeichen-Reihenfolge für Handbücher
uv run chunking --write docs/chunks.md	Chunks als Markdown ablegen

Netzbasierte Quellen

Eine Quelle wird am Präfix src: erkannt und nicht über core/paths.py aufgelöst — eine Adresse ist kein Pfad und darf nicht gegen das Dokumentenverzeichnis geprüft werden. Ohne --allow-remote fragt die Anwendung nach, bevor sie das Netz anfasst.

- src:https://example.com/handbuch — Webseite als Text
- src:wikipedia:RAG — Wikipedia-Artikel zum Thema
- src:csv:https://example.com/d.csv — entfernte CSV-Datei
- Alternativ als Option: --url, --wikipedia, --csv-url, jeweils mehrfach angebbbar

Reproduzierbarkeit

- Netzbasierte Loader sind nicht deterministisch: Derselbe Aufruf kann zu zwei Zeitpunkten verschiedene Inhalte liefern. Für einen wiederholbaren Index legt `--load-jsonl` den Ladesatz ab, bevor bereinigt und gechunkt wird. Ein zweiter Lauf nimmt ihn mit `--from-jsonl` wieder auf.

RAG-Anwendung

Die RAG-Anwendung ist der einzige Einstieg, der einen Schlüssel und das Netz braucht. Die Stufen eins bis drei laufen lokal, der Index lebt im Arbeitsspeicher und wird bei jedem Lauf neu aufgebaut.

Aufruf	Wirkung
<code>uv run rag</code>	Übersicht: Ladebericht, Reinigungsbilanz, Chunking, Vorschau
<code>uv run rag --config</code>	aktive Einstellungen ohne Klartext-Keys
<code>uv run rag --load</code>	Stufe 1: Ladebericht
<code>uv run rag --clean</code>	Stufe 2: Reinigungsbilanz
<code>uv run rag --chunks</code>	Stufe 3: Chunking-Kennzahlen
<code>uv run rag --preview</code>	die ersten Chunks mit Herkunft
<code>uv run rag --retrieve "Frage"</code>	Treffer zeigen, ohne das Modell zu rufen
<code>uv run rag --ask "Frage"</code>	eine Frage beantworten
<code>uv run rag --ask "Frage"</code> <code>--show-hits</code>	Antwort mit Belegstellen
<code>uv run rag --compare</code>	alle Testfragen in beiden Suchmodi

Option	Wirkung	Standard
<code>--language de en</code>	Sprache der Wissensbasis	de
<code>--format pdf txt</code>	Quellformat	pdf
<code>--mode dense hybrid</code>	Suchmodus	dense

Option	Wirkung	Standard
--k N	Zahl der Treffer	3
--chunk-size N	Zielgröße eines Chunks	800
--chunk-overlap N	Überlappung zweier Chunks	100
--strategy NAME	recursive, character oder token	recursive
--embedding-model NAME	Modell der Indexierung	text-embedding-3-small
--temperature N	Temperatur des antwortenden Modells	0.0

Testbefehle

Die Tests laufen ohne Netz und ohne echte Schlüssel; Geheimnisse werden als Umgebungsvariablen gesetzt. Für die Abdeckung muss der Pfad genau auf `src/rag_project` zeigen, weil das Paket dort liegt.

- `uv run pytest` — die gesamte Suite
- `uv run pytest --cov=src/rag_project --cov-report=term-missing -q` — Abdeckung im Terminal
- `uv run pytest --cov=src/rag_project --cov-report=html -q` — HTML-Bericht unter `htmlcov/`
- `uv sync --group dev` — Testwerkzeuge nachinstallieren

Typischer Arbeitsablauf

Ein vollständiger Durchlauf von der Einrichtung bis zur beantworteten Frage sieht so aus.

- `uv sync --group dev` — alles installieren
- `uv run pytest` — Suite vor der Änderung laufen lassen
- `uv run rag-project --check` — Konfiguration und Modell prüfen
- `uv run rag --chunks` — Vorverarbeitung ohne Netz kontrollieren
- `uv run rag --ask "FRAGE" --show-hits` — eine Frage mit Belegstellen beantworten